Tsw

Programmierhandbuch (deutsch)

2026-10-27

Inhalt

index **3**
toggleswitch **3**
tsw **11**
Lizenz **18**

index

Stichwörter: index

Das Kippschalter-Widget-Paket Tsw 1.5

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

Tsw-Programmierhandbuch

Der Befehl tsw::toggleswitch

toggleswitch

Stichwörter: toggleswitch

Der Befehl `tsw::toggleswitch`

Für Tsw Version 1.5

von

Csaba Nemethi
csaba.nemethi@t-online.de

Inhalt

- Kurzreferenz
- Ausführliche Referenz
- Wahl zwischen toggleswitch und (ttk::)checkbutton

Kurzreferenz

NAME

`tsw::toggleswitch` – Kippschalter-Widgets erzeugen und bedienen

ÜBERSICHT

```
tsw::toggleswitch pathName ?options?
```

STANDARDOPTIONEN

```
-class       -style         -underline
-cursor      -takefocus     -width
-font        -text
-foreground  -textvariable
```

WIDGET-SPEZIFISCHE OPTIONEN

```
-command command

-offvalue value

-onvalue value

-size 1|2|3

-variable variable
```

WIDGET-BEFEHL

```
pathName attrib ?name ?value name value ...??

pathName cget option

pathName configure ?option ?value option value ...??

pathName hasattrib name

pathName identify ?element? x y

pathName instate stateSpec ?script?

pathName state ?stateSpec?

pathName style

pathName switchstate ?boolean?

pathName toggle

pathName unsetattrib name
```

SCHLÜSSELWÖRTER

toggleswitch, trough, slider, widget

Ausführliche Referenz

NAME

`tsw::toggleswitch` – Kippschalter-Widgets erzeugen und bedienen

ÜBERSICHT

```
tsw::toggleswitch pathName ?options?
```

BESCHREIBUNG

Der Befehl `tsw::toggleswitch` erzeugt ein neues Fenster namens `pathName` und macht es zu einem **toggleswitch**-Widget. Weitere Optionen, die unten beschrieben sind, können auf der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Eigenschaften des toggleswitch-Widgets festzulegen, etwa seine Größe und das Tcl-Skript, das jedes Mal läuft,

wenn der Schaltzustand des Widgets umgeschaltet wird. Der Befehl `tsw::toggleswitch` gibt sein Argument `pathName` zurück. Wenn dieser Befehl aufgerufen wird, darf noch kein Fenster namens `pathName` existieren, das Elternfenster von `pathName` muss aber existieren.

Ein toggleswitch ist ein Megawidget, das im Wesentlichen aus einer waagerechten **Laufschiene** und einem **Schieber** besteht, genau wie ein ttk::scale-Widget. Diese Elemente gehören tatsächlich zu einem ttk::scale, das im Widget enthalten ist. Die Laufschiene ist ein vollständig abgerundetes gefülltes Rechteck, und der Schieber ist ein gefüllter Kreis oder ein vollständig abgerundetes gefülltes Rechteck in der Laufschiene. Beide Elemente werden mit skalierungsbewussten SVG-Bildern gezeichnet. Ihre Abmessungen hängen vom Wert der Konfigurationsoption `-size` ab, vom aktuellen Theme, von der Skalierungsstufe der Anzeige und vom Skalierungsfaktor von Tk, den `[tk scaling]` zum Zeitpunkt angibt, an dem das erste toggleswitch-Widget erzeugt wird. Das Widget enthält außerdem ein ttk::label, das einen optionalen Text links von der Skala anzeigt; dieses Kindwidget wird nur verwaltet, wenn der mit der Option `-text` oder `-textvariable` angegebene Text eine nichtleere Zeichenkette ist.

Wie ein Lichtschalter kann ein toggleswitch-Widget einen von zwei möglichen **Schaltzuständen** haben: ein oder aus. Im Zustand ein liegt der Schieber am Ende der Laufschiene, und im Zustand aus an ihrem Anfang. Wie der Abschnitt VORGABEBINDUNGEN unten beschreibt, schaltet der Benutzer mit der Maus oder der Taste `space` zwischen diesen beiden Zuständen um.

Der Tcl-Befehl, der zu einem toggleswitch-Widget gehört, hat eine sehr einfache API. Mit dem Unterbefehl `switchstate` ändern oder fragen Sie den Schaltzustand des Widgets ab. Mit der Konfigurationsoption `-command` geben Sie ein Skript an, das jedes Mal läuft, wenn der Schaltzustand des Widgets umgeschaltet wird. Wie (ttk::)checkbutton unterstützen toggleswitch-Widgets auch die Optionen `-offvalue`, `-onvalue` und `-variable`.

Die Farben, mit denen Laufschiene und Schieber in den verschiedenen Widget-Zuständen gezeichnet werden (etwa `active`, `background`, `disabled`, `pressed` und `selected`), hängen vom aktuellen Theme ab. Die Implementierung erzeugt diese Elemente bei Bedarf mit Hilfe allgemeiner Prozeduren für beliebige Themes und themespezifischer Prozeduren für einige eingebaute Themes. Anwendungen können für ein beliebiges Theme `theme` *ausdrückliche* Unterstützung hinzufügen, indem sie einen passenden Befehl namens `tsw::createElements_theme` bereitstellen.

STANDARDOPTIONEN

```
-class       -style         -underline
-cursor      -takefocus     -width
-font        -text
-foreground  -textvariable
```

Einzelheiten zu den Standardoptionen stehen im Handbucheintrag **ttk_widget**. Der Vorgabewert der Option `-class` ist `Toggleswitch`. Der Wert der Option `-style` ist voreingestellt auf `Toggleswitch2`, passend zum Vorgabewert `2` der Option `-size`. Je nach

dem Wert der Option `-size` ist der Standardwert von `-style` einer der Werte `Toggleswitch1`, `Toggleswitch2` oder `Toggleswitch3` (siehe unten). Der Vorgabewert der Option `-takefocus` ist `"ttk::takefocus"` (wie bei den meisten Ttk-Widgets).

WIDGET-SPEZIFISCHE OPTIONEN

Befehlszeilenname:	-command
Name der Datenbank:	command
Datenbankklasse:	Command

Legt ein Tcl-Skript fest, das im globalen Geltungsbereich ausgewertet wird, jedes Mal wenn der Schaltzustand des Widgets umgeschaltet wird (programmgesteuert, durch den Aufruf des Unterbefehls `switchstate` oder `toggle`, oder interaktiv). Die Vorgabe ist eine leere Zeichenkette.

Befehlszeilenname:	-offvalue
Name der Datenbank:	offValue
Datenbankklasse:	OffValue

Der Wert, der in der zugehörigen Variablen gespeichert wird, wenn der Schaltzustand des Widgets auf aus gesetzt wird. Voreingestellt ist `0`.

Befehlszeilenname:	-onvalue
Name der Datenbank:	onValue
Datenbankklasse:	OnValue

Der Wert, der in der zugehörigen Variablen gespeichert wird, wenn der Schaltzustand des Widgets auf ein gesetzt wird. Voreingestellt ist `1`.

Befehlszeilenname:	-size
Name der Datenbank:	size
Datenbankklasse:	Size

Legt den Größenbezeichner des toggleswitch-Widgets fest. Die unterstützten Werte sind die Zeichenketten `1`, `2` (die Vorgabe) und `3`. Ist das aktuelle Theme `aqua`, entsprechen diese Größenbezeichner den Steuergrößen "mini", "small" und "large" der nativen Kippschalter unter macOS. Für macOS bis Version 15 (Sequoia) steht der Wert `1` für die Laufschienengröße von 26 x 15 Pixeln, der Wert `2` für die Laufschienengröße von 32 x 18 Pixeln, und der Wert `3` bezeichnet die Laufschienengröße von 38 x 22 Pixeln. Ab macOS 26 (Tahoe) betragen die Laufschienengrößen 29 x 13, 36 x 16 und 44 x 20 Pixel. Bei allen anderen Themes steht auf

einem unskalierten Bildschirm der Wert 1 für die Laufschienengröße von 32 x 16 Pixeln, der Wert 2 für die Laufschienengröße von 40 x 20 Pixeln, und der Wert 3 bezeichnet die Laufschienengröße von 48 x 24 Pixeln. Ausnahme: unter Windows 10 und früher ist aus Kompatibilität mit dem nativen Kippschalter bei den Themes `vista`, `winnative` und `xpnative` die unskalierte Laufschienenbreite 35, 44 und 53 Pixel.

Beachten Sie: Wird diese Option auf einen Wert `size` gesetzt, wechselt der Wert der Option `-style` automatisch zu `Toggleswitchsize`, wenn sein bisheriger oder angeforderter Wert einer der Werte `Toggleswitch1`, `Toggleswitch2` oder `Toggleswitch3` war, und zu `prefix.Toggleswitchsize`, wenn sein bisheriger oder angeforderter Wert einer der Werte `prefix.Toggleswitch1`, `prefix.Toggleswitch2` oder `prefix.Toggleswitch3` war.

Umgekehrt wird die Option `-size` automatisch auf `size` gesetzt, wenn die Option `-style` auf einen Wert der Form `Toggleswitchsize` oder `prefix.Toggleswitchsize` gesetzt wird (wobei `size` einer der Werte `1`, `2` oder `3` ist).

Werden beide Optionen `-size` und `-style` konfiguriert, hat die erste Vorrang vor der zweiten, unabhängig von der Reihenfolge, in der sie angegeben wurden. Wenn Sie das Widget zum Beispiel so erzeugen

```
tsw::toggleswitch pathName -size 3 -style My.Toggleswitch3
```

oder aufrufen

```
pathName configure -style My.Toggleswitch3
```

dann wird die Option `-style` auf `My.Toggleswitch3` gesetzt (und die Option `-size` hat den Wert `3`). Wenn Sie das Widget dagegen so erzeugen

```
tsw::toggleswitch pathName -style My.Toggleswitch3
```

dann hat die Option `-style` den Wert `My.Toggleswitch2` und nicht `My.Toggleswitch3`, weil die Erzeugung des Widgets die Vorgabe `-size 2` auslöst, die Vorrang vor `-style My.Toggleswitch3` hat.

Befehlszeilenname:	-variable
Name der Datenbank:	variable
Datenbankklasse:	Variable

Der Name einer globalen Variablen, deren Wert mit dem toggleswitch verknüpft ist. Der Schaltzustand des Widgets wechselt auf ein, wenn diese Variable auf den Wert gesetzt wird, den die Option `-onvalue` angibt, und sonst auf aus. Ist nichts angegeben, ist die Vorgabe der Pfadname des Widgets.

Beachten Sie: Wie bei (ttk::)checkbutton führt das Umschalten des Schaltzustands durch eine Änderung des Wertes dieser Variablen *nicht* dazu, dass das von der Option `-command` angegebene Skript ausgeführt wird.

WIDGET-BEFEHL

Der Befehl `tsw::toggleswitch` erzeugt einen neuen Tcl-Befehl, dessen Name `pathName` ist. Mit diesem Befehl rufen Sie verschiedene Operationen auf dem Widget auf. Er hat die

folgende allgemeine Form:

```
pathName option ?arg arg ...?
```

`option` und die Argumente `arg` bestimmen das genaue Verhalten des Befehls. Für toggleswitch-Widgets sind die folgenden Befehle möglich:

pathName **attrib**?`name ?value name value ...??`
Fragt die Attribute des Widgets ab oder ändert sie. Ist kein `name` angegeben, gibt der Befehl eine Liste von Paaren zurück, von denen jedes den Namen und den Wert eines Attributs für `pathName` enthält. Ist `name` ohne `value` angegeben, gibt der Befehl den Wert des genannten Attributs zurück, oder eine leere Zeichenkette, wenn kein entsprechender Wert existiert (mit dem Unterbefehl `hasattrib` unterscheiden Sie diesen Fall von dem, in dem der Wert eines *vorhandenen* Attributs eine leere Zeichenkette ist). Sind ein oder mehrere Paare aus `name` und `value` angegeben, setzt der Befehl die genannten Widget-Attribute auf die genannten Werte; in diesem Fall ist der Rückgabewert eine leere Zeichenkette. `name` kann eine beliebige Zeichenkette sein.

pathName **cget** `option`
Gibt den aktuellen Wert der Konfigurationsoption zurück, die `option` angibt. Sie kann jeden Wert haben, den der Befehl `tsw::toggleswitch` akzeptiert.

pathName **configure**?`option ?value option value ...??`
Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Ist keine `option` angegeben, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für `pathName` beschreibt (das Format dieser Liste steht bei `Tk_ConfigureInfo`). Ist `option` ohne `value` angegeben, gibt der Befehl eine Liste zurück, die die genannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Wertes, der zurückgegeben wird, wenn keine `option` angegeben ist). Sind ein oder mehrere Paare aus `option` und `value` angegeben, ändert der Befehl die genannten Widget-Optionen auf die genannten Werte; in diesem Fall ist der Rückgabewert eine leere Zeichenkette. `option` kann jeden Wert haben, den der Befehl `tsw::toggleswitch` akzeptiert.

pathName **hasattrib** `name`
Gibt `1` zurück, wenn das Attribut `name` vorhanden ist, andernfalls `0`.

pathName **identify**?`element? x y`
Siehe den Handbucheintrag zu **ttk_widget**. Die Koordinaten werden relativ zum toggleswitch erwartet, sie werden aber in Werte relativ zum Kind ttk::scale umgerechnet, und der Rückgabewert ist der Name des entsprechenden Stilelements dieses Kindes oder eine leere Zeichenkette.

pathName **instate** `stateSpec ?script?`
Siehe den Handbucheintrag zu **ttk_widget**.

pathName **state**?`stateSpec?`
Siehe den Handbucheintrag zu **ttk_widget**.

pathName **style**
Gibt den Stil zurück, den das zugrunde liegende ttk::scale-Widget verwendet. Bei Ttk-Widgets wurde dieser Unterbefehl in Tk 8.7a4 eingeführt, das toggleswitch-Widget stellt ihn aber für alle unterstützten Tk-Versionen bereit.

pathName **switchstate**?`boolean?`
Ändert den Schaltzustand des Widgets oder fragt ihn ab. Ist das optionale Argument

vorhanden, muss es ein boolescher Wert sein (ein numerischer Wert, bei dem `0` falsch ist und alles andere wahr, oder eine Zeichenkette wie `true/yes/on` oder `false/no/off`). Ist das Zustandsflag `disabled` des Widgets gesetzt, gibt der Befehl unmittelbar nach der Prüfung des Arguments eine leere Zeichenkette zurück. Sonst setzt der Befehl, wenn das Argument wahr ist, den Schaltzustand des Widgets auf ein, indem er das Flag `selected` des zugrunde liegenden ttk::scale-Widgets setzt, den Schieber an das Ende der Laufschiene bewegt und die zugehörige Variable auf den Wert setzt, den die Option `-onvalue` angibt. Ist das Argument falsch, setzt der Befehl den Schaltzustand des Widgets auf aus, indem er das Flag `selected` löscht, den Schieber an den Anfang der Laufschiene bewegt und die zugehörige Variable auf den Wert setzt, den die Option `-offvalue` angibt.

Wenn der Wert des Arguments den Schaltzustand des Widgets umschaltet und das Skript, das als Wert der Option `-command` angegeben ist, eine nichtleere Zeichenkette ist, wertet der Befehl dieses Skript im globalen Geltungsbereich aus und gibt dessen Ergebnis zurück; sonst ist der Rückgabewert eine leere Zeichenkette.

Ist das optionale Argument nicht vorhanden, gibt der Befehl den aktuellen Schaltzustand des Widgets als `0` (aus) oder `1` (ein) zurück. Wenn ein toggleswitch-Widget erzeugt wird, wird sein Schaltzustand mit `0` initialisiert.

`pathName` **toggle**
Dieser bequeme Unterbefehl schaltet den Schaltzustand des Widgets um. Er entspricht logisch dem Aufruf des Befehls `switchstate` mit dem Argument `0`, wenn der aktuelle Schaltzustand ein ist, und sonst mit dem Argument `1`.

`pathName` **unsetattrib** `name`
Hebt das Attribut `name` auf. Gibt eine leere Zeichenkette zurück.

VORGABEBINDUNGEN

Das Paket Tsw ersetzt die Vorgabebindungen des ttk::scale, das in einem toggleswitch-Widget enthalten ist, durch eigene Bindungen, und zwar so:

Ist das aktuelle Theme `aqua`:

1. Wird die Maustaste 1 über dem Schieber gedrückt und die Maus dann bei gehaltener Taste 1 gezogen, bis der Zeiger die Laufschiene betritt, bewegt sich der Schieber gleichmäßig zur gegenüberliegenden Kante der Laufschiene, und der Schaltzustand des Widgets wird umgeschaltet. Dasselbe geschieht, wenn die Maustaste 1 außerhalb des Schiebers gedrückt wird und der Zeiger das Widget dann bei gehaltener Taste 1 waagerecht verlässt.
2. Wird die Maustaste 1 irgendwo im Widget gedrückt und dann über dem Widget losgelassen, ohne dass der Schieber vorher bewegt wurde, bewegt sich dieser gleichmäßig zur gegenüberliegenden Kante der Laufschiene, und der Schaltzustand des Widgets wird umgeschaltet.
3. Hat das Widget den Eingabefokus, schaltet die Taste `space` seinen Schaltzustand um.

Ist das aktuelle Theme nicht `aqua`:

1. Wird die Maustaste 1 irgendwo im Widget gedrückt und die Maus dann bei gehaltener Taste 1 gezogen, bewegt sich der Schieber in derselben (waagerechten) Richtung wie der Zeiger. Beim Loslassen der Taste wird der Schaltzustand auf aus oder ein gesetzt, je nach der Lage

des Schiebers relativ zur Mitte des Widgets.

2. Wird die Maustaste 1 irgendwo im Widget gedrückt und dann über dem Widget losgelassen, ohne dass die Maus vorher waagerecht gezogen wurde, wird der Schaltzustand des Widgets umgeschaltet.
3. Hat das Widget den Eingabefokus, schaltet die Taste `space` seinen Schaltzustand um.

Ist das Zustandsflag `disabled` des Widgets gesetzt, tritt keine der oben genannten Aktionen ein.

WIDGET-ZUSTÄNDE

Das Widget setzt den Zustand `selected`, jedes Mal wenn der Schaltzustand auf ein wechselt, und löscht ihn sonst. Die Vorgabebindungen setzen und löschen die Zustandsflags `active` und `pressed`.

STILOPTIONEN

Der Vorgabeklassenname für ein `tsw::toggleswitch` ist `Toggleswitch`.

Dynamische Zustände: `active`, `background`, `disabled`, `pressed`, `selected`.

Stiloptionen, die mit `ttk::style` für `Toggleswitch1`, `Toggleswitch2` und `Toggleswitch3` eingestellt werden können:

-padding

padding

Vorgabe ist

0.75p

.

Das ttk::scale und das ttk::label, das einen optionalen Text anzeigt, sind Kinder eines ttk:frame mit dem Layout `ToggleswitchHull`, das für alle Themes außer `aqua` ein Element `focus` enthält. Stiloptionen, die mit `ttk::style` für `ToggleswitchHull` eingestellt werden können:

-focuscolor

color

Die Vorgabe hängt vom Theme ab.

-focussolid

boolean

Vorgabe ist true für das Theme

classic

und false für alle anderen. Diese Option wurde in Tk 8.6.15 eingeführt; in früheren Tk-Versionen hat sie keine Wirkung.

-focusthickness

amount

Die Vorgabe ist

1

. Der Wert kann jede Form haben, die

Tk_GetPixels

akzeptiert.

Beim Theme aqua werden diese fokusbezogenen Optionen einfach ignoriert.

Wie ttk-Stile konfiguriert werden, steht auf der Handbuchseite **ttk_style**. SCHLÜSSELWÖRTER toggleswitch, trough, slider, widget

---

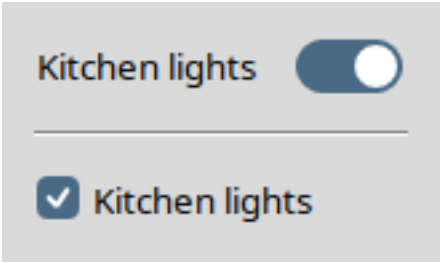
Wahl zwischen toggleswitch und (ttk::)checkbutton

Dieser Abschnitt beruht auf dem Microsoft-Learn-Artikel *Toggle switches*.

Für manche Aktionen kann ein toggleswitch oder ein checkbutton passen. Um zu entscheiden, welches Steuerelement besser passt, folgen Sie diesen Hinweisen:

- Verwenden Sie einen toggleswitch für binäre Einstellungen, die wie Aktionen wirken und sofort gelten, nachdem der Benutzer den Schalter umlegt.

  Im folgenden Beispiel sollten Sie zum Einschalten der Küchenbeleuchtung einen toggleswitch verwenden und keinen checkbutton.



Vergleich

- Für optionale Einträge ("nice to have") verwenden Sie checkbutton.
- Verwenden Sie checkbutton, wenn der Benutzer mehrere Einträge auswählen kann, die zu einer einzigen Einstellung oder Funktion gehören.
- Verwenden Sie einen checkbutton, wenn der Benutzer weitere Schritte ausführen muss, damit Änderungen wirksam werden. Wenn der Benutzer zum Beispiel eine Schaltfläche "Submit" oder "Next" anklicken muss, um Änderungen anzuwenden, verwenden Sie einen checkbutton.

tsw

Stichwörter: tsw

Tsw-Programmierhandbuch

Für Tsw Version 1.5

von

Csaba Nemethi

csaba.nemethi@t-online.de

Inhalt

Überblick
- Was ist Tsw?
- Woher bekomme ich es?
- Wie installiere ich es?
- Wie verwende ich es?

Beispiele
- Tsw-Demo
- Tablelist-Bearbeitungsoptionen

Überblick

Was ist Tsw?

Tsw steht für **T**oggle **sw**itch (Kippschalter) und ist ein Bibliothekspaket für Tcl/Tk ab Version 8.6. Ist die Version 8.6, muss sich zusätzlich die Erweiterung tksvg in den Interpreter laden lassen (Tk ab 8.7 und 9.0 hat SVG fest eingebaut). Das Paket ist in reinem Tcl/Tk geschrieben und enthält:

- die Implementierung des Kippschalter-Megawidgets toggleswitch, einschließlich eines allgemeinen Hilfsmoduls für Megawidgets;
- zwei ausführlich kommentierte Demo-Skripte mit den typischen Schritten, um toggleswitch-Widgets zu erzeugen und zu bedienen;
- dieses Tutorial;
- eine Referenzseite im HTML-Format.

Ein toggleswitch ist ein Megawidget, das im Wesentlichen aus einer waagerechten Laufschiene (einem vollständig abgerundeten gefüllten Rechteck) und einem Schieber (einem gefüllten Kreis oder einem vollständig abgerundeten gefüllten Rechteck in der Laufschiene) besteht. Das Widget enthält außerdem ein ttk::label, das einen optionalen Text links von der Laufschiene anzeigt; dieses Kindwidget wird nur verwaltet, wenn der mit der Option `-text` oder `-textvariable` angegebene Text eine nichtleere Zeichenkette ist.

Ein toggleswitch-Widget kann einen von zwei Schaltzuständen haben: ein oder aus. Im Zustand ein liegt der Schieber am Ende der Laufschiene, und im Aus-Zustand an ihrem Anfang. Der Benutzer schaltet mit der Maus oder der Leertaste zwischen diesen beiden Zuständen um.

Mit dem Unterbefehl `switchstate` des Tcl-Befehls, der zu einem toggleswitch gehört, ändern oder fragen Sie den Schaltzustand des Widgets ab. Mit der Konfigurationsoption `-command` geben Sie ein Skript an, das jedes Mal läuft, wenn der Schaltzustand des Widgets umgeschaltet wird. Zur Kompatibilität mit (ttk::)checkbutton unterstützen toggleswitch-Widgets auch die Optionen `-offvalue`, `-onvalue` und `-variable`.

Woher bekomme ich es?

Tsw steht auf der Webseite zum kostenlosen Herunterladen bereit

https://www.nemethi.de

Die Distributionsdatei heißt `tsw1.5.tar.gz` für UNIX und `tsw1_5.zip` für Windows. Diese Dateien enthalten dieselben Angaben, abgesehen von dem zusätzlichen Wagenrücklaufzeichen vor dem Zeilenvorschub am Ende jeder Zeile in den Textdateien für Windows.

Tsw ist auch in tklib enthalten, unter der Adresse

https://core.tcl-lang.org/tklib

Wie installiere ich es?

Installieren Sie das Paket als Unterverzeichnis eines der Verzeichnisse, die die Variable `auto_path` angibt. Sie können es zum Beispiel als Unterverzeichnis des Verzeichnisses `lib` in Ihrer Tcl/Tk-Installation ablegen (auf derselben Ebene wie das Unterverzeichnis `tk8.X` oder `tk9.X`).

Um Tsw *unter UNIX* zu installieren, wechseln Sie mit `cd` in das gewünschte Verzeichnis und packen die Distributionsdatei `tsw1.5.tar.gz` aus:

```
gunzip -c tsw1.5.tar.gz | tar -xf -
```

Auf den meisten UNIX-Systemen kann dies durch Folgendes ersetzt werden:

```
tar -zxf tsw1.5.tar.gz
```

Beide Befehle legen ein Verzeichnis namens `tsw1.5` an, mit den Unterverzeichnissen `demos`, `doc` und `scripts`.

Unter Windows packen Sie mit WinZip oder einem anderen Programm die Distributionsdatei `tsw1_5.zip` in das Verzeichnis `tsw1.5` aus, mit den Unterverzeichnissen `demos`, `doc` und `scripts`.

Beachten Sie, dass tklib das Tsw-Verzeichnis `demos` durch das Unterverzeichnis `tsw` des Verzeichnisses `examples` ersetzt. Bitte berücksichtigen Sie das, wenn Sie die Beispiele unten lesen.

Wie verwende ich es?

Damit Ihre Skripte die Befehle und Variablen des Pakets Tsw erreichen, müssen sie eine der Zeilen enthalten

```
package require tsw ?version?
package require Tsw ?version?
```

Sie können jede der beiden Anweisungen oben verwenden, weil die Datei `tsw.tcl` beide Zeilen enthält

```
package provide tsw ...
package provide Tsw ...
```

Sie können eine dieser beiden Zeilen aus `tsw.tcl` entfernen, wenn das Paket sich nicht unter zwei verschiedenen Namen bekannt machen soll. Damit beschränken Sie das Argument von `package require` auf einen einzigen Namen.

Da das Paket Tsw in einem eigenen Namespace namens `tsw` implementiert ist, führen Sie entweder den Befehl

```
namespace import tsw::toggleswitch
```

aus, um die einzige öffentliche *Prozedur* des Namespace `tsw` zu importieren, oder Sie verwenden den qualifizierten Namen `tsw::toggleswitch`. Die Beispiele unten verwenden den zweiten Weg.

Um auf Tsw-*Variablen* zuzugreifen, *müssen* Sie qualifizierte Namen verwenden. Nur zwei

Tsw-Variablen sind dafür vorgesehen, außerhalb des Namespace `tsw` verwendet zu werden:

- Die Variable `tsw::version` enthält die aktuelle Versionsnummer des Pakets Tsw.
- Die Variable `tsw::library` enthält den Ort des Tsw-Installationsverzeichnisses.

Beispiele

Tsw-Demo

Das Skript `TswDemo.tcl` im Verzeichnis `demos` erzeugt vier toggleswitch-Widgets und zeigt, wie ihr Aussehen vom aktuellen Theme abhängt. Das Theme wählen Sie über das Menü, das zu einem ttk::menubutton-Widget gehört. Außerdem zeigt es, wie Sie ein Skript angeben, das jedes Mal läuft, wenn der Schaltzustand eines toggleswitch umgeschaltet wird.



Hier ist der Code, der die vier toggleswitch-Widgets erzeugt:

```
package require Tk
package require tsw

wm title . "Tsw Demo"

ttk::frame .tf
ttk::frame .bf

#
# Create 3 toggleswitch widgets having different values of the -size option
#
set l1 [ttk::label .tf.l1 -text "Toggle switch of size 1"]
set sw1 [tsw::toggleswitch .tf.sw1 -size 1]
set l2 [ttk::label .tf.l2 -text "Toggle switch of size 2"]
set sw2 [tsw::toggleswitch .tf.sw2 -size 2]
$sw2 switchstate 1
set l3 [ttk::label .tf.l3 -text "Toggle switch of size 3"]
set sw3 [tsw::toggleswitch .tf.sw3 -size 3]

#
# Create a toggleswitch widget of default size and set its -command option
#
set l4 [ttk::label .tf.l4 -text "Enable/disable above widgets"]
set sw4 [tsw::toggleswitch .tf.sw4]
$sw4 switchstate 1
$sw4 configure -command [list toggleWidgetsState $sw4]
```

Die toggleswitch-Widgets entstehen durch den Aufruf des Befehls `tsw::toggleswitch`. Bei den ersten drei toggleswitch-Widgets wird die Option `-size` auf `1`, `2` und `3` gesetzt. Beim letzten toggleswitch wird diese Option nicht ausdrücklich gesetzt, sie hat also ihren Vorgabewert 2. Wie die Bildschirmfotos zeigen, hängen beim Theme `aqua` die Farben der

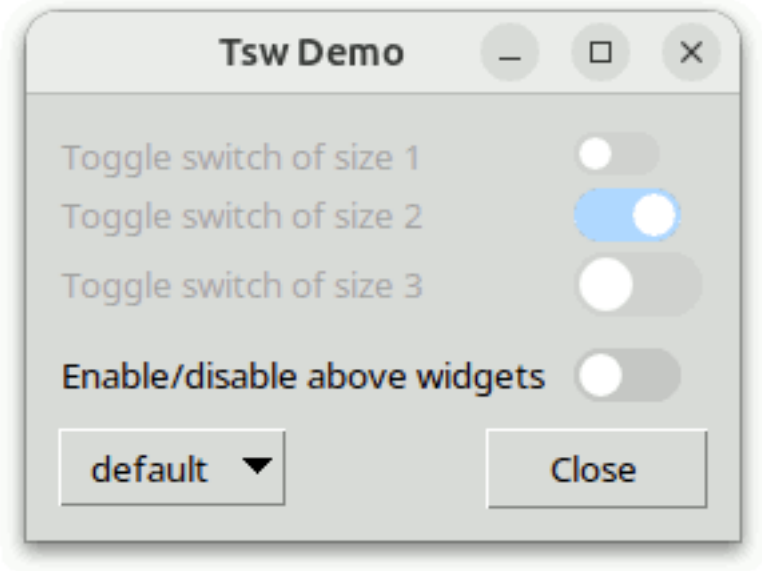
toggleswitch-Widgets auch vom Erscheinungsbild des Systems ab (heller oder dunkler Modus) und von der Akzentfarbe. Die ersten beiden Bildschirmfotos stammen von macOS 15 (Sequoia), das dritte von macOS 26 (Tahoe).

Bei zwei der vier toggleswitch-Widgets wechselt der Schaltzustand vom Vorgabewert `0` (aus) auf `1` (ein), indem der Unterbefehl `switchstate` des zugehörigen Tcl-Befehls aufgerufen wird. Zusätzlich wird beim letzten toggleswitch die Option `-command` auf ein Skript gesetzt, das jedes Mal läuft, wenn der Schaltzustand des Widgets umgeschaltet wird. Dieses Skript ruft die Prozedur `toggleWidgetsState` auf, die so implementiert ist:

```
#------------------------------------------------------------------------------
# toggleWidgetsState
#
# Enables/disables the widgets in the first 3 grid rows, depending on the
# switch state of the specified toggleswitch widget.
#------------------------------------------------------------------------------
proc toggleWidgetsState sw {
    global l1 l2 l3 sw1 sw2 sw3
    set stateSpec [expr {[$sw switchstate] ? "!disabled" : "disabled"}]
    foreach w [list $l1 $l2 $l3 $sw1 $sw2 $sw3] {
        $w state $stateSpec
    }
}
```

Diesmal wird der Unterbefehl `switchstate` ohne das optionale Argument aufgerufen und gibt deshalb den aktuellen Schaltzustand des toggleswitch-Widgets zurück.

Beim Theme `default` sieht das Fenster, nachdem der Schaltzustand des letzten toggleswitch von ein auf aus gewechselt hat, aus wie im Bildschirmfoto unten:



Tsw-Demo

Der Rest des Codes gehört nicht zu Tsw und wird deshalb hier nicht gezeigt.

Tablelist-Bearbeitungsoptionen

Das Skript `EditingOpts.tcl` im Verzeichnis `demos` ist eine leicht angepasste Fassung des Tablelist-Demo-Skripts `tileWidgets2.tcl`. Es zeigt die interaktive Zellenbearbeitung von

tablelist mit verschiedenen Ttk-Widgets und dem toggleswitch-Widget. Die zusätzliche Funktion in dieser Fassung steckt in den Prozeduren `configEditing` und `applySwitchState`. Die erste, ausgelöst durch die Schaltfläche "Configure Editing", öffnet ein Toplevel-Fenster mit toggleswitch-Widgets, um die bearbeitungsbezogenen tablelist-Optionen mit booleschen Werten einzustellen, die Tablelist-Benutzer im Lauf der Jahre vorgeschlagen haben. So lässt sich bequem prüfen, welche Wirkung das Setzen und Löschen dieser booleschen Optionen hat.





Die Prozedur `configEditing` steht unten:

```
package require Tk
package require tsw
package require tablelist_tile

. . .

#------------------------------------------------------------------------------
# configEditing
#
# Configures the editing-related tablelist options having boolean values with
# the aid of toggleswitch widgets.
#------------------------------------------------------------------------------
proc configEditing tbl {
    set top .top
    if {[winfo exists $top]} {
        raise $top
        focus $top
        return ""
    }

    toplevel $top
    wm title $top "Editing Options"

    set tf [ttk::frame $top.tf]
    set bf [ttk::frame $top.bf]

    #
    # Create the widgets corresponding to the
    # editing-related options with boolean values
    #
    set row 0
    foreach opt {
        -autofinishediting
        -editendonfocusout
        -editendonmodclick
        -editselectedonly
        -forceeditendcommand
        -instanttoggle
        -showeditcursor
    } {
```

```
        lassign [$tbl configure $opt] option dbName dbClass default current
        set defaultStr [expr {$default ? "on" : "off"}]

        set sw [tsw::toggleswitch $tf.sw$row -text "$opt ($defaultStr)"]
        if {$current != $default} {
            $sw configure -foreground red2
        }

        $sw switchstate $current        ;# sets the switch state to $current
        $sw attrib default $default     ;# saves $default as attribute value
        $sw configure -command [list applySwitchState $sw $tbl $opt]
        pack $sw -padx 9p -pady {0 3p} -fill x

        incr row
    }

    . . .
}
```

Für jede der 7 bearbeitungsbezogenen Optionen mit booleschen Werten zeigt die Prozedur den Namen der Option und den Vorgabewert in einem toggleswitch-Widget an und setzt dessen Schaltzustand auf den aktuellen Wert der Option. Außerdem ruft sie den Unterbefehl `attrib` des Tcl-Befehls auf, der zum toggleswitch gehört, und speichert den Vorgabewert als Wert des Widget-Attributs mit dem Namen `default`. So "merkt" sich das Widget den Vorgabewert auf objektorientierte Weise und kann ihn später abrufen, ohne externe Mittel zu brauchen. Zur besseren Bedienung erscheint der Text des toggleswitch in der Vordergrundfarbe `red2`, wenn der aktuelle Wert der Option vom Vorgabewert abweicht.

Jedes Mal, wenn der Schaltzustand des toggleswitch-Widgets umgeschaltet wird, ruft das Skript, das als Wert der Option `-command` angegeben ist, die Prozedur `applySwitchState` auf, die unten steht:

```
#-------------------------------------------------------------------------
# applySwitchState
#
# Sets the configuration option opt of the tablelist tbl and the foreground
# color of the toggleswitch widget sw according to its switch state.
#-------------------------------------------------------------------------
proc applySwitchState {sw tbl opt} {
    set switchState [$sw switchstate]
    $tbl configure $opt $switchState

    set fgColor [expr {$switchState == [$sw attrib default] ? "" : "red2"}]
    $sw configure -foreground $fgColor
}
```

Die angegebene tablelist-Option wird auf den Schaltzustand des toggleswitch-Widgets gesetzt, und ebenso die Vordergrundfarbe des Widgets, abhängig vom Schaltzustand und vom Vorgabewert der Option. Den Vorgabewert holt der Unterbefehl `attrib` des toggleswitch.

Beachten Sie, dass in diesem Beispiel die Texte des Fensters "Editing Options" die Werte der Option `-text` der toggleswitch-Widgets sind. Im vorigen Beispiel stehen dafür zusätzliche ttk::label-Widgets links neben den zugehörigen toggleswitch-Widgets.

Lizenz

Deutsche Übersetzung © 2026 Gregor Ebbing.

Die Seiten wurden aus den Originalen nach Markdown umgewandelt und ins Deutsche übersetzt; Auszeichnung ohne Platz im Format (Verweise ohne Ziel, Layoutbilder) kann dabei wegfallen. Das sind die Änderungen; für Teile unter der Mozilla Public License 1.1 (tDOM) ist es die Beschreibung der Änderungen nach Abschnitt 3.3.

Die Übersetzung entstand mit Unterstützung von DeepL und KI-Werkzeugen und wurde von Hand durchgesehen und überarbeitet.

Die Übersetzung steht unter denselben Bedingungen wie das jeweilige englische Original. Diese Bedingungen folgen. Die Copyright-Angaben der Originalseiten stehen bei jeder Seite.

Toggle switch widget package Tsw 1.5 Copyright (c) 2025-2026 Csaba Nemethi (E-mail: csaba.nemethi@t-online.de)

This library is free software; you can use, modify, and redistribute it for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions.

This software is distributed WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Stichwortverzeichnis

A

attrib 8

C

cget 8
configure 8

H

hasattrib 8

I

identify 8
index 3
instate 8

S

state 8
STILOPTIONEN 10
style 8
switchstate 8

T

Tablelist-Bearbeitungsoptionen 15
toggle 9
toggleswitch 3
tsw 11
Tsw-Demo 14

U

unsetattrib 9

V

VORGABEBINDUNGEN 9

W

Wahl zwischen toggleswitch und (ttk::)checkbutton 11
WIDGET-ZUSTÄNDE 10